

Feature Envy

```
public class TimelineView {  
  
    public String habitDateViewAll(HabitTracker ht){  
        List<Habit> habits = ht.getHabits();  
        StringBuilder response = new StringBuilder();  
        for(Habit habit : habits){  
            response.append("[ Habit: ")  
                .append(habit.getName())  
                .append(" Records: ");  
            List<LocalDateTime> records =  
                ht.getHabitRecords(habit.getId());  
            for(LocalDateTime record : records){  
                response.append(ht.formatHabitDate(record)).append(", ");  
            }  
            response.append("]");  
        }  
  
        return response.toString();  
    }  
  
}
```

Refatore o code smell Feature envy utilizando a técnica de refatoração move to:

```
public class HabitTracker {  
    public class TimelineView {  
  
        public String habitDateViewAll(HabitTracker ht){  
            List<Habit> habits = ht.getHabits();  
            StringBuilde ↓ ponse = new StringBuilder();  
            for(Habit habit : habits){
```

```

        response.append("%s".formatHabitDate(record["append"], "?"))
    }
    response.append("]");
}

return response.toString();
}

// Instance == null
if (instance == null) {
    instance = new HabitTracker();
}
return instance;
}

private HabitTracker() {
    this.habits = new ArrayList<>();
    this.tracker = new HashMap<>();
    this.nextId = 1;
}

@Override
public String toString() {
    StringBuilder response = new StringBuilder();
    for (Habit habit : habits) {
        response.append(habit.toString() + append", "?);
    }
    return "Habit: " + response.toString();
}

public Habit getHabitById(Integer id) {
    return this.habits.stream()
        .filter(habit -> Objects.equals(habit.getId(), id))
        .findFirst().orElse(null);
}

public List<Habit> getHabit() {
    return this.habits;
}

public String formatHabitDate(LocalDate date) {
    DateTimeFormatter formatter =
        DateTimeFormatter.ofPattern("ddMMyyyyHHmmss");
    return date.format(formatter);
}

public List<Integer> getTrackerKeys() {
    return this.tracker.keySet().stream().toList();
}

public int addHabit(String name, String motivation, Integer
    dayOfWeek, Integer dedication, Integer dayOfMonth, Integer year,
    Integer month, Integer day, Integer hour, Integer minute, Integer
    second, Boolean isConcluded) {
    LocalDateTime l = LocalDateTime.of(dayOfWeek, dedication,
        dayOfMonth, minute, second);
    LocalDateTime startDate = LocalDateTime.of(year, month, day,
        hour, minute, second);
    Habit habit = new Habit(name, motivation, l, startDate,
        startDate, isConcluded);
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}

public int handleAddHabitAdapter(List<String> stringProperties,
    List<Integer> intProperties, Boolean isConcluded) {
    return addHabit(stringProperties.get(0), stringProperties.get(1),
        intProperties.get(2), intProperties.get(3), intProperties.get(4),
        intProperties.get(5), intProperties.get(6), intProperties.get(7),
        intProperties.get(8), intProperties.get(9), isConcluded);
}

public int addHabit(String name, String motivation) {
    Habit habit = new Habit(name, motivation, this.nextId);
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}

public void addHabitRecord(Integer id) {
    tracker.get(id).add(LocalDate.now());
}

public void toggleConcludeHabit(Integer id) {
    for (Habit habit : this.habits) {
        if (habit.getId() == id) {
            habit.setIsConcluded(!habit.getIsConcluded());
        }
    }
}

public void removeHabit(Integer id) {
    this.habits.removeIf(habit -> habit.getId() == id);
    this.tracker.remove(id);
}

public List<LocalDateTime> getHabitRecords(Integer id) {
    return this.tracker.get(id);
}

public List<String> searchHabit(String search) {
    List<String> habits = new ArrayList<>();
    for (Habit habit : this.habits) {
        if (habit.getName().toLowerCase().contains(search.toLowerCase()) ||
            habit.getMotivation().toLowerCase().contains(search.toLowerCase())) {
            habits.add(habit.toString());
        }
    }
    return habits;
}
}

```



O código contém um code smell do tipo **Feature Envy**, onde o método `habitDateViewAll` da classe `TimelineView` está muito dependente de funcionalidades da classe `HabitTracker`. Para corrigir isso, utilizaremos a técnica de refatoração **Move Method**, movendo as responsabilidades para as classes apropriadas. Isso garantirá que as operações relacionadas a hábitos e seus registros sejam tratadas pela classe `HabitTracker` ou até pela classe `Habit`.

Segue o código refatorado:

</> Refactor Feature Envy

```
1  public class TimelineView {
2
3  public String habitDateViewAll(HabitTracker ht) {
4      return ht.getFormattedHabitsWithRecords();
5  }
6  }
7
8  public class HabitTracker {
9
10     // Restante do código da classe HabitTracker...
11
12     public String getFormattedHabitsWithRecords() {
13         StringBuilder response = new StringBuilder();
14         for (Habit habit : this.habits) {
```

Explicação das mudanças:

1. Movimento de Lógica para `HabitTracker`:

- O método `getFormattedHabitsWithRecords` foi adicionado à classe `HabitTracker`. Ele encapsula a lógica de iteração pelos hábitos e pela formatação das informações relacionadas aos registros.

2. Delegação para `Habit`:



A formatação de cada registro de um hábito e seus registros foi movida para a classe `Habit`.

Explicação das mudanças:

1. Movimento de Lógica para `HabitTracker`:

- O método `getFormattedHabitsWithRecords` foi adicionado à classe `HabitTracker`. Ele encapsula a lógica de iteração pelos hábitos e pela formatação das informações relacionadas aos registros.

2. Delegação para `Habit`:

- A formatação específica de um hábito e seus registros foi movida para a classe `Habit` por meio do método `getFormattedHabitWithRecords`. Isso segue o princípio de responsabilidade única, pois a classe `Habit` agora lida com detalhes relacionados a si mesma.

3. Uso de `Function` para Flexibilidade:

- O método `getFormattedHabitWithRecords` usa uma interface funcional (`Function<LocalDateTime, String>`) para formatar as datas. Isso mantém o método genérico e independente de implementações específicas.

Essas mudanças resultam em um design mais coeso, eliminando o code smell **Feature Envy** e promovendo a manutenção do código.


```

private List<String> handleRegistrySearch(String text){
    List<String> results = new ArrayList<>();

    results.addAll(CardManager.getInstance().searchCards(text));
    results.addAll(HabitTracker.getInstance().searchHabits(text));
    results.addAll(TodoTracker.getInstance().searchTodos(text));

    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRe
gistry(text));
    this.searchLog.addSearchHistory(text);
    this.searchLog.setNumUsages(this.searchLog.getNumUsages() +
1);
    results.add("logged in: " + this.searchLog.getLogName());
    return results;
}

public class SearchLog {
    private List<String> searchHistory;
    private Map<String, Integer> searchCount;
    private boolean isLocked;
    private Integer numUsages;
    private String logName;

    public SearchLog(String logName) {
        searchHistory = new ArrayList<>();
        searchCount = new HashMap<>();
        this.logName = logName;
        numUsages = 0;
        isLocked = false;
    }

    public void addSearchHistory(String searchHistory) {
        this.searchHistory.add(searchHistory);
    }

    public List<String> getSearchHistory() {
        return searchHistory;
    }

    public void setSearchHistory(List<String> searchHistory) {
        this.searchHistory = searchHistory;
    }

    public Map<String, Integer> getSearchCount() {
        return searchCount;
    }

    public void setSearchCount(Map<String, Integer> searchCount) {
        this.searchCount = searchCount;
    }

    public boolean isLocked() {
        return isLocked;
    }

    public void setLocked(boolean locked) {
        isLocked = locked;
    }

    public Integer getNumUsages() {
        return numUsages;
    }

    public void setNumUsages(Integer numUsages) {
        this.numUsages = numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setLogName(String logName) {
        this.logName = logName;
    }
}

refatore o code small Feature envy code utilizando a tecnica de
refatoração move method

```

java

 Copiar código

```
public class SearchLog {
    private List<String> searchHistory;
    private Map<String, Integer> searchCount;
    private boolean isLocked;
    private Integer numUsages;
    private String logName;

    public SearchLog(String logName) {
        searchHistory = new ArrayList<>();
        searchCount = new HashMap<>();
        this.logName = logName;
        numUsages = 0;
        isLocked = false;
    }

    public void addSearchHistory(String searchHistory) {
        this.searchHistory.add(searchHistory);
    }

    public List<String> getSearchHistory() {
        return searchHistory;
    }

    public void setSearchHistory(List<String> searchHistory) {
        this.searchHistory = searchHistory;
    }
}
```

```
public Map<String, Integer> getSearchCount() {  
    return searchCount;  
}  
  
public void setSearchCount(Map<String, Integer> searchCount) {  
    this.searchCount = searchCount;  
}  
  
public boolean isLocked() {  
    return isLocked;  
}  
  
public void setLocked(boolean locked) {  
    isLocked = locked;  
}  
  
public Integer getNumUsages() {  
    return numUsages;  
}  
  
public void setNumUsages(Integer numUsages) {  
    this.numUsages = numUsages;  
}  
  
public String getLogName() {  
    return logName;  
}  
  
public void setLogName(String logName) {  
    this.logName = logName;  
}  
}
```

 Copiar código



java

 Copiar código

```
private List<String> handleRegistrySearch(String text) {
    List<String> results = new ArrayList<>();
    results.addAll(CardManager.getCardManager().searchInCards(text));
    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));
    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

    // Agora, a responsabilidade de adicionar ao histórico de buscas é da classe SearchLog
    this.searchLog.addSearchHistory(text);
    this.searchLog.setNumUsages(this.searchLog.getNumUsages() + 1);

    results.add("\nLogged in: " + this.searchLog.getLogName());
    return results;
}
```



eu quero que voce mova

```
private List<String> handleRegistrySearch(String text){
    List<String> results = new ArrayList<>();

    results.addAll(CardManager.getCardManager().searchInCards(text));

    results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
    results.addAll(TodoTracker.getInstance().searchInTodos(text));

    results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));
    this.searchLog.addSearchHistory(text);
    this.searchLog.setNumUsages(this.searchLog.getNumUsages() + 1);
    results.add("\nLogged in: " + this.searchLog.getLogName());
    return results;
}
```

para Searchlog

java

 Copiar código

```
public class SearchLog {
    private List<String> searchHistory;
    private Map<String, Integer> searchCount;
    private boolean isLocked;
    private Integer numUsages;
    private String logName;

    public SearchLog(String logName) {
        searchHistory = new ArrayList<>();
        searchCount = new HashMap<>();
        this.logName = logName;
        numUsages = 0;
        isLocked = false;
    }

    // Método movido para a classe SearchLog
    public List<String> handleRegistrySearch(String text) {
        List<String> results = new ArrayList<>();
        results.addAll(CardManager.getCardManager().searchInCards(text));
        results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
        results.addAll(TodoTracker.getInstance().searchInTodos(text));
        results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

        this.addSearchHistory(text); // Agora o método addSearchHistory pertence à
        this.setNumUsages(this.getNumUsages() + 1); // Atualiza o número de usos

        results.add("\nLogged in: " + this.getLogName());
        return results;
    }
}
```

```
// Métodos existentes de SearchLog
public void addSearchHistory(String searchHistory) {
    this.searchHistory.add(searchHistory);
}

public List<String> getSearchHistory() {
    return searchHistory;
}

public void setSearchHistory(List<String> searchHistory) {
    this.searchHistory = searchHistory;
}

public Map<String, Integer> getSearchCount() {
    return searchCount;
}

public void setSearchCount(Map<String, Integer> searchCount) {
    this.searchCount = searchCount;
}

public boolean isLocked() {
    return isLocked;
}

public void setLocked(boolean locked) {
    isLocked = locked;
}

public Integer getNumUsages() {
    return numUsages;
}
```

 Copiar código

```
}

public void setNumUsages(Integer numUsages) {
    this.numUsages = numUsages;
}

public String getLogName() {
    return logName;
}

public void setLogName(String logName) {
    this.logName = logName;
}
}
```

Classe Original (ajustada para chamar o método de `SearchLog`)

Agora, a classe que anteriormente possuía o método `handleRegistrySearch` irá chamar o método dentro de `SearchLog` para realizar as buscas e registrar as entradas no histórico:

java

 Copiar código

```
// Supondo que 'searchLog' seja uma instância de SearchLog
List<String> results = this.searchLog.handleRegistrySearch(text);
```

Justificativa da Refatoração

- O método `handleRegistrySearch` foi movido para a classe `SearchLog`, e agora é a classe `SearchLog` que gerencia tanto as buscas quanto a atualização do histórico.
- A classe que usava esse método (originalmente onde ele estava) simplesmente invoca o método da classe `SearchLog`, o que reduz o acoplamento e melhora a coesão da classe `SearchLog`, que agora é responsável por todas as operações relacionadas ao registro e histórico de buscas.



como ficaria esse código sem essa classe agora?

```
package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TODOTracker;
import org.example.studyregistry.StudyTaskManager;

import java.util.ArrayList;
import java.util.List;

public class RegistrySearch implements Search<String>{
    private SearchLog searchLog = new SearchLog("Registry Search");
    public RegistrySearch(){}

    @Override
    public List<String> search(String text) {
        return handleRegistrySearch(text);
    }

    public SearchLog getSearchLog() {
        return searchLog;
    }

    private List<String> handleRegistrySearch(String text){
        List<String> results = new ArrayList<>();

        results.addAll(CardManager.getCardManager().searchInCards(text));

        results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
        results.addAll(TODOTracker.getInstance().searchInTodos(text));

        results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));
        this.searchLog.addSearchHistory(text);
        this.searchLog.setNumUsages(this.searchLog.getNumUsages() + 1);
        results.add("\nLogged in: " + this.searchLog.getLogName());
        return results;
    }
}
```


java

 Copiar código

```
package org.example.studysearch;

import org.example.studycards.CardManager;
import org.example.studyplanner.HabitTracker;
import org.example.studyplanner.TODOTracker;
import org.example.studyregistry.StudyTaskManager;

import java.util.ArrayList;
import java.util.List;

public class RegistrySearch implements Search<String> {
    private SearchLog searchLog = new SearchLog("Registry Search");

    public RegistrySearch() {}

    @Override
    public List<String> search(String text) {
        return searchLog.handleRegistrySearch(text);
    }

    public SearchLog getSearchLog() {
        return searchLog;
    }
}
```

Alterações feitas:

- **Remoção do método `handleRegistrySearch` de `RegistrySearch`:** O método `handleRegistrySearch` foi removido de `RegistrySearch` e movido para a classe `SearchLog`. Agora, o método é chamado diretamente pela instância de `SearchLog` dentro de `RegistrySearch`.
- **`search()` agora chama `searchLog.handleRegistrySearch()`:** Em vez de chamar um método próprio para lidar com a busca, a classe `RegistrySearch` chama diretamente o método `handleRegistrySearch` da instância `searchLog`.

Justificativa:

- **Responsabilidade concentrada:** A classe `SearchLog` agora é responsável pela busca e pelo registro de histórico de buscas, tornando o código de `RegistrySearch` mais simples e com responsabilidades bem definidas.
- **Coesão e baixo acoplamento:** A refatoração melhora a coesão da classe `SearchLog`, que gerencia tanto as buscas quanto o histórico. `RegistrySearch` agora se limita a delegar a responsabilidade para `SearchLog`, mantendo a separação de preocupações.

🔊 📄 👍 🗨️ ↺ ~

poderia retirar search log desse código deixar apenas coisas ligadas ao registrysearch?

Alterações feitas:

1. **Remoção de `searchlog`:** O `searchlog` foi removido, e agora a classe `megasearcher` gerencia diretamente o histórico de buscas (`searchhistory`) e a contagem de usos (`usage`).
2. **Responsabilidade do Histórico:** A responsabilidade de adicionar aos históricos de buscas (`addsearchhistory`) foi mantida dentro da classe `megasearcher`.
3. **Contagem de Usos:** A contagem de quantas vezes a busca foi realizada agora é gerida diretamente em `megasearcher`, através do campo `usage`.
4. **Gerenciamento do nome do log:** A classe `megasearcher` agora lida com o nome do log diretamente através do campo `logfile`.

Justification:

- **Consolidação das responsabilidades:** A classe `Agendamento` agora é a única responsável tanto pela busca quanto pelo gerenciamento do histórico e contagem de usos, eliminando qualquer dependência externa.
- **Simplicidade:** A refatoração simplifica a arquitetura, tornando `Agendamento` completamente autônoma, sem depender de outra classe (`Usuario`) para gerenciar o estado do histórico ou da contagem de buscas.

```
private List<String> handleMaterialSearch(String text){
    List<String> results = new ArrayList<>();

    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
    this.searchLog.addSearchHistory(text);
    this.searchLog.setNumUsages(this.searchLog.getNumUsages() + 1);
    results.add("\nLogged in: " + this.searchLog.getLogName());
    return results;
}
```

```
public class SearchLog {
    private List<String> searchHistory;
    private Map<String, Integer> searchCount;
    private boolean isLocked;
    private Integer numUsages;
    private String logName;
```

```
    public SearchLog(String logName) {
        searchHistory = new ArrayList<>();
        searchCount = new HashMap<>();
        this.logName = logName;
        numUsages = 0;
        isLocked = false;
    }
```

```
// Método movido para a classe SearchLog
public List<String> handleRegistrySearch(String text) {
    List<String> results = new ArrayList<>();
```

```

results.addAll(CardManager.getCardManager().searchInCards(text));

results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
results.addAll(TodoTracker.getInstance().searchInTodos(text));

results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

this.addSearchHistory(text); // Agora o método addSearchHistory
pertence à classe SearchLog
this.setNumUsages(this.getNumUsages() + 1); // Atualiza o
número de usos

results.add("\nLogged in: " + this.getLogName());
return results;
}

// Métodos existentes de SearchLog
public void addSearchHistory(String searchHistory) {
    this.searchHistory.add(searchHistory);
}

public List<String> getSearchHistory() {
    return searchHistory;
}

public void setSearchHistory(List<String> searchHistory) {
    this.searchHistory = searchHistory;
}

public Map<String, Integer> getSearchCount() {
    return searchCount;
}

```

```

    }

    public Map<String, Integer> getSearchCount() {
        return searchCount;
    }

    public void setSearchCount(Map<String, Integer> searchCount) {
        this.searchCount = searchCount;
    }

    public boolean isLocked() {
        return isLocked;
    }

    public void setLocked(boolean locked) {
        isLocked = locked;
    }

    public Integer getNumUsages() {
        return numUsages;
    }

    public void setNumUsages(Integer numUsages) {
        this.numUsages = numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setName(String logName) {
        this.logName = logName;
    }

```

Envie uma mensagem para o ChatGPT

```

        return numUsages;
    }

    public void setNumUsages(Integer numUsages) {
        this.numUsages = numUsages;
    }

    public String getLogName() {
        return logName;
    }

    public void setLogName(String logName) {
        this.logName = logName;
    }
}

refatore o code smell Feature envy code utilizando a técnica de
refatoração move method

```

Método de Coesão.

Aqui está a refatoração sugerida:

Antes da refatoração:

```
java Copiar código

private List<String> handleMaterialSearch(String text){
    List<String> results = new ArrayList<>();
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));
    this.searchLog.addSearchHistory(text);
    this.searchLog.setNumUsages(this.searchLog.getNumUsages() + 1);
    results.add("\nLogged in: " + this.searchLog.getLogName());
    return results;
}
```

Depois da refatoração:

```
java Copiar código

public class SearchLog {
    private List<String> searchHistory;
    private Map<String, Integer> searchCount;
    private boolean isLocked;
    private Integer numUsages;
    private String logName;

    public SearchLog(String logName) {
        searchHistory = new ArrayList<>();
        searchCount = new HashMap<>();
        this.logName = logName;
        numUsages = 0;
        isLocked = false;
    }

    // Método movido para a classe SearchLog
    public List<String> handleRegistrySearch(String text) {
        List<String> results = new ArrayList<>();
        results.addAll(CardManager.getCardManager().searchInCards(text));
        results.addAll(HabitTracker.getHabitTracker().searchInHabits(text));
        results.addAll(TodoTracker.getInstance().searchInTodos(text));
        results.addAll(StudyTaskManager.getStudyTaskManager().searchInRegistries(text));

        this.addSearchHistory(text);
        this.setNumUsages(this.getNumUsages() + 1); // Atualiza o número de usos

        results.add("\nLogged in: " + this.getLogName());
        return results;
    }
}
```

```
// Métodos existentes de SearchLog
public void addSearchHistory(String searchHistory) {
    this.searchHistory.add(searchHistory);
}

public List<String> getSearchHistory() {
    return searchHistory;
}

public void setSearchHistory(List<String> searchHistory) {
    this.searchHistory = searchHistory;
}

public Map<String, Integer> getSearchCount() {
    return searchCount;
}

public void setSearchCount(Map<String, Integer> searchCount) {
    this.searchCount = searchCount;
}

public boolean isLocked() {
    return isLocked;
}

public void setLocked(boolean locked) {
    isLocked = locked;
}

public Integer getNumUsages() {
    return numUsages;
}
```

 Copiar código

```
public void setNumUsages(Integer numUsages) {
    this.numUsages = numUsages;
}

public String getLogName() {
    return logName;
}

public void setLogName(String logName) {
    this.logName = logName;
}
}
```


Código de `handleMaterialSearch` após a refatoração:

java

 Copiar código

```
private List<String> handleMaterialSearch(String text) {  
    List<String> results = new ArrayList<>();  
    results.addAll(StudyMaterial.getStudyMaterial().searchInMaterials(text));  
    results.addAll(this.searchLog.handleRegistrySearch(text)); // Chama o método mo  
    return results;  
}
```

O que foi alterado:

1. O método `handleRegistrySearch` foi movido para a classe `SearchLog`, que agora gerencia

O que foi alterado:

1. O método `handleRegistrySearch` foi movido para a classe `SearchLog`, que agora gerencia tanto o histórico de pesquisa quanto o aumento do contador de usos diretamente.
2. A classe que utilizava o `SearchLog` agora chama `handleRegistrySearch` diretamente da instância de `SearchLog`, simplificando a interação com a classe.

Essa refatoração elimina a dependência da classe externa no método de pesquisa, melhora a coesão da classe `SearchLog` e segue a técnica de "Move Method", reduzindo o acoplamento entre as classes.

```

public String getRandomCardFromBox() {
    String response = "";
    response += letterSystem.getMethodName();
    List<Box> boxes = letterSystem.getBoxes();
    response += letterSystem.getRandomCard(boxes);
    return response;
}

public class LetterSystem extends StudyMethod {
    List<Box> boxes = null;

    public LetterSystem(String methodName) {
        super(methodName);
        boxes = new ArrayList<>();
        boxes.add(new Box(), new Box(), new Box(), new Box(), new Box());
    }

    @Override
    public String getMethodName() {
        return this.methodName;
    }

    @Override
    void setMethodName(String methodName) {
        this.methodName = methodName;
    }

    @Override
    public String toString() {
        StringBuffer response = new StringBuffer();
        int index = 0;
        for(Box box : boxes) {
            response.append("Box ");
            response.append(index).append(" ");
            index++;
        }
        return response.toString();
    }

    public void clearBoxes() {
        boxes.clear();
        boxes = new ArrayList<>();
        boxes.add(new Box(), new Box(), new Box(), new Box(), new Box());
    }

    public List<Box> getBoxes() {
        return boxes;
    }

    public String getRandomCard(List<Box> otherBoxes) {
        if(otherBoxes == null) {
            return null;
        }
        if(otherBoxes.isEmpty()) {
            return null;
        }
        Box allBoxes = new Box();
        for(Box box : otherBoxes) {
            allBoxes.addCard(box.getCards());
        }
        Integer randomCard = allBoxes.getRandomCard();
        if(randomCard == null) {
            return "No card found";
        }
        CardManager manager = CardManager.getCardManager();
        Card card = manager.getCard().getCard();
        String response = "[" + randomCard + "]";
        response += "The random question was: " + card.getQuestion() +
            "\n";
        response += "The answer is: " + card.getAnswer();
        return response;
    }

    public void addCardToBox(Integer id, Integer cardId) {
        this.boxes.get(id).addCard(cardId);
    }

    public void removeCardFromBox(Integer id, Integer cardId) {
        this.boxes.get(id).removeCard(cardId);
    }

    public Card takeCardFromBox(Integer boxId) {
        Integer cardId = boxes.get(boxId).getRandomCard();
        return this.cardManager.getCard(cardId);
    }

    public void boxValidation(Integer boxId) throws Exception {
        if(boxId == null || boxId > (boxes.size() - 1) || boxId < 0) {
            throw new Exception("Invalid box ID");
        }
    }

    public void upgradeCard(Integer cardId, Integer boxId) throws
        Exception {
        boxValidation(boxId);

        Box refBox = boxes.get(boxId);
        if(refBox.hasCard(cardId)) {
            throw new Exception("No card found");
        }
        refBox.removeCard(cardId);
        boxes.get(Math.min(boxId + 1, 4)).addCard(cardId);
    }

    public void downgradeCard(Integer cardId, Integer boxId) throws
        Exception {
        boxValidation(boxId);

        Box refBox = boxes.get(boxId);
        if(refBox.hasCard(cardId)) {
            throw new Exception("No card found");
        }
        refBox.removeCard(cardId);
        boxes.get(Math.max(boxId - 1, 0)).addCard(cardId);
    }
}

refatore o code small Feature envoy code utilizando a tecnica de
refatoração move method

```

```
public class LeitnerSystem extends StudyMethod {
    List<Box> boxes = null;

    public LeitnerSystem(String methodName) {
        super(methodName);
        boxes = new ArrayList<>(Arrays.asList(new Box(), new Box(), new Box(), new
    }

    @Override
    public String getMethodName() {
        return this.methodName;
    }

    @Override
    void setMethodName(String methodName) {
        this.methodName = methodName;
    }

    @Override
    public String toString() {
        StringBuilder response = new StringBuilder();
        int index = 0;
        for (Box box : boxes) {
            response.append("Box ").append(index).append(": ").append(box.toString()
            index++;
        }
        return response.toString();
    }

    public void clearBoxes() {
        boxes.clear();
        boxes = new ArrayList<>(Arrays.asList(new Box(), new Box(), new Box(), new
```

[Copiar código](#)

 Copiar código

```
public void clearBoxes() {
    boxes.clear();
    boxes = new ArrayList<>(Arrays.asList(new Box(), new Box(), new Box(), new
}

public List<Box> getBoxes() {
    return boxes;
}

public String getRandomCardFromBox() {
    String response = "";
    response += getMethodName();
    response += getRandomCard(boxes);
    return response;
}

public String getRandomCard(List<Box> otherBoxes) {
    if (otherBoxes == null || otherBoxes.isEmpty()) {
        return null;
    }
    Box allBoxes = new Box();
    for (Box box : otherBoxes) {
        allBoxes.addCards(box.getCards());
    }
    Integer randomCard = allBoxes.getRandomCard();
    if (randomCard == null) {
        return "No card found";
    }
    CardManager manager = CardManager.getCardManager();
    Card card = manager.getCard(randomCard);
    String response = "[" + randomCard + " ] ";
    response += "The random question was: " + card.getQuestion() + " ! !";
}
```

```
    CardManager manager = CardManager.getCardManager();
    Card card = manager.getCard(randomCard);
    String response = "[" + randomCard + "] ";
    response += "The random question was: " + card.getQuestion() + " | ";
    response += "The answer is: " + card.getAnswer();
    return response;
}

public void addCardToBox(Integer id, Integer boxId) {
    this.bboxes.get(boxId).addCard(id);
}

public void removeCardFromBox(Integer id, Integer boxId) {
    this.bboxes.get(boxId).removeCard(id);
}

public Card takeCardFromBox(Integer boxId) {
    Integer cardId = bboxes.get(boxId).getRandomCard();
    return this.cardManager.getCard(cardId);
}

public void boxIdValidation(Integer boxId) throws Exception {
    if (boxId == null || boxId > (bboxes.size() - 1) || boxId <= 0) {
        throw new Exception("Invalid box ID");
    }
}

public void upgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);

    Box refBox = bboxes.get(boxId);
    if (refBox.hasCard(cardId)) {
```

 Copiar código

```

        throw new Exception("Invalid box ID");
    }
}

public void upgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);

    Box refBox = boxes.get(boxId);
    if (refBox.hasCard(cardId)) {
        throw new Exception("No card Found");
    }
    refBox.removeCard(cardId);
    boxes.get(Math.min(boxId + 1, 4)).addCard(cardId);
}

public void downgradeCard(Integer cardId, Integer boxId) throws Exception {
    boxIdValidation(boxId);

    Box refBox = boxes.get(boxId);
    if (refBox.hasCard(cardId)) {
        throw new Exception("No card Found");
    }
    refBox.removeCard(cardId);
    boxes.get(Math.max(boxId - 1, 0)).addCard(cardId);
}
}

```

 Copiar código

Classe `StudyMethod` (ou onde o método estava originalmente):

Remova o método `getRandomCardFromBox` da classe `StudyMethod`, pois ele foi movido para `LeitnerSystem`.



java

 Copiar código

```

        if (refBox.hasCard(cardId)) {
            throw new Exception("No card Found");
        }
        refBox.removeCard(cardId);
        boxes.get(Math.max(boxId - 1, 0)).addCard(cardId);
    }
}

```

 Copiar código

Classe `StudyMethod` (ou onde o método estava originalmente):

Remova o método `getRandomCardFromBox` da classe `StudyMethod`, pois ele foi movido para `LeitnerSystem`.

java

 Copiar código

```
// Remover método getRandomCardFromBox da classe StudyMethod
```

Justificativa:

- A refatoração melhora a coesão da classe `LeitnerSystem`, que agora lida com a lógica de obter um cartão aleatório de uma caixa diretamente, sem depender de um método externo.
- O método `getRandomCardFromBox` faz mais sentido dentro de `LeitnerSystem`, já que ele depende de acessar as caixas e utilizar a lógica de randomização associada diretamente à implementação de `LeitnerSystem`.

como vai ficar isso aqui agora?

```

public void handleGetRandomCardFromBox() {
    ↓ try{
        String response = getRandomCardFromBox();
    }
}

```

```
public void handleGetRandomCardFromBox() {
    try{
        String response = getRandomCardFromBox();
        System.out.println(response);
    }catch (Exception e){
        System.out.println(e.getMessage());
    }
}
```

Após a refatoração, o método `handleGetRandomCardFromBox()` precisará ser ajustado para garantir que ele está chamando o método correto. Como o método `getRandomCardFromBox()` foi movido para a classe `LeitnerSystem`, agora você precisará garantir que o método seja chamado a partir de uma instância dessa classe.

Aqui está como o código pode ficar após a refatoração:

Código refatorado para `handleGetRandomCardFromBox`:

java

 Copiar código

```
public void handleGetRandomCardFromBox(LeitnerSystem leitnerSystem) {
    try {
        // Chama o método getRandomCardFromBox da instância de LeitnerSystem
        String response = leitnerSystem.getRandomCardFromBox();
        System.out.println(response);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
```




```
// Chama o método getRandomCardFromBox da instância de LeitnerSystem
String response = leitnerSystem.getRandomCardFromBox();
System.out.println(response);
} catch (Exception e) {
    System.out.println(e.getMessage());
}
}
```

 Copiar código

Alterações:

- **Parâmetro** `LeitnerSystem`: O método agora recebe uma instância de `LeitnerSystem` como parâmetro para acessar o método `getRandomCardFromBox()`. Isso é necessário porque a função foi movida para dentro de `LeitnerSystem`.

Exemplo de uso:

Agora, para usar `handleGetRandomCardFromBox`, você deve passar uma instância de `LeitnerSystem` ao chamá-la:

java

 Copiar código

```
LeitnerSystem leitnerSystem = new LeitnerSystem("Leitner Method");
handleGetRandomCardFromBox(leitnerSystem);
```

Isso garantirá que a lógica de obtenção do cartão aleatório seja executada corretamente.

```
void assignActions(){
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
    actions.put("↓", this::handleRemoveCard);
    actions.put("4", this::handleRandomFlashCard);
}
```

Isso garantirá que a lógica de obtenção do cartão aleatório seja executada corretamente.

```
void assignActions(){
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
    actions.put("3", this::handleRemoveCard);
    actions.put("4", this::handleRandomFlashCard);
    actions.put("5", this::handleInsertCardInBox);
    actions.put("6", this::handleRemoveCardFromBox);
    actions.put("7", this::handleUpgradeCardFromBox);
    actions.put("8", this::handleDowngradeCardFromBox);
    actions.put("9", this::handleViewBoxes);
    actions.put("10", this::handleGetRandomCardFromBox);
}
```

```
java: incompatible types: invalid method reference
  method handleGetRandomCardFromBox in class
  org.example.controllers.StudyCardsController cannot be applied to
  given types
    required: org.example.studycards.LeitnerSystem
    found:    no arguments
    reason: actual and formal argument lists differ in length
```

O erro ocorre porque o método `handleGetRandomCardFromBox()` agora precisa de um argumento do tipo `LeitnerSystem`, mas você está tentando chamá-lo sem argumentos ao usá-lo na referência de método. O Java está reclamando disso porque não é possível passar o método sem os parâmetros necessários para a execução.

Aqui estão duas formas de resolver isso:



Opção 1: Alterar a assinatura do método `handleGetRandomCardFromBox` para não exigir argumentos

Se for possível que o método `handleGetRandomCardFromBox` acesse a instância de `LeitnerSystem` sem precisar passar explicitamente como argumento, você pode modificar a assinatura desse método para acessar a instância de `LeitnerSystem` de alguma forma (por exemplo, através de uma variável de instância ou contexto).

Exemplo:

Modificando `handleGetRandomCardFromBox` para acessar `LeitnerSystem` internamente:

```
java Copiar código

public void handleGetRandomCardFromBox() {
    try {
        // Acesse LeitnerSystem diretamente
        String response = this.leitnerSystem.getRandomCardFromBox(); // Supondo que
        System.out.println(response);
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
```

Ajuste do método `assignActions`:

Agora, você pode manter a referência de método sem argumentos, porque `handleGetRandomCardFromBox()` não precisa mais de parâmetros:

```
java Copiar código

void assignActions(){
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
}
```

Ajuste do metodo `assignActions`:

Agora, você pode manter a referência de método sem argumentos, porque `handleGetRandomCardFromBox()` não precisa mais de parâmetros:

java

 Copiar código

```
void assignActions(){
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
    actions.put("3", this::handleRemoveCard);
    actions.put("4", this::handleRandomFlashCard);
    actions.put("5", this::handleInsertCardInBox);
    actions.put("6", this::handleRemoveCardFromBox);
    actions.put("7", this::handleUpgradeCardFromBox);
    actions.put("8", this::handleDowngradeCardFromBox);
    actions.put("9", this::handleViewBoxes);
    actions.put("10", this::handleGetRandomCardFromBox); // Aqui agora está corret
}
```

Opção 2: Passar `LeitnerSystem` como argumento para o método

`handleGetRandomCardFromBox`

Se você preferir que o método `handleGetRandomCardFromBox()` continue recebendo o argumento `LeitnerSystem`, você pode usar uma **lambda expression** ao invés de uma referência de método direta para passar o argumento.

Usando uma `lambda` para passar o argumento:

java

 Copiar código

```
void assignActions() {
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
```

Opção 2: Passar `LeitnerSystem` como argumento para o método `handleGetRandomCardFromBox`

Se você preferir que o método `handleGetRandomCardFromBox()` continue recebendo o argumento `LeitnerSystem`, você pode usar uma **lambda expression** ao invés de uma referência de método direta para passar o argumento.

Usando uma **lambda** para passar o argumento:

java

 Copiar código

```
void assignActions() {
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
    actions.put("3", this::handleRemoveCard);
    actions.put("4", this::handleRandomFlashCard);
    actions.put("5", this::handleInsertCardInBox);
    actions.put("6", this::handleRemoveCardFromBox);
    actions.put("7", this::handleUpgradeCardFromBox);
    actions.put("8", this::handleDowngradeCardFromBox);
    actions.put("9", this::handleViewBoxes);
    actions.put("10", (key) -> handleGetRandomCardFromBox(leitnerSystem)); // Usar
}
```

Neste caso, você está criando uma expressão lambda que chama `handleGetRandomCardFromBox(leitnerSystem)` ao ser invocada, passando `leitnerSystem` como argumento.

Resumo:

- Se o método `handleGetRandomCardFromBox()` puder acessar `LeitnerSystem` internamente, a primeira opção é mais simples.



java

 Copiar código

```
package org.example.controllers;

import org.example.studycards.LeitnerSystem;

public class StudyCardsController {

    private LeitnerSystem leitnerSystem;

    public StudyCardsController(LeitnerSystem leitnerSystem) {
        this.leitnerSystem = leitnerSystem;
    }

    public void handleGetRandomCardFromBox() {
        try {
            // Acesse LeitnerSystem diretamente
            String response = this.leitnerSystem.getRandomCardFromBox(); // Supondo
            System.out.println(response);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    // O método assignActions agora pode ser usado para registrar as ações
    public void assignActions() {
        actions.put("1", this::handleViewCards);
        actions.put("2", this::handleCreateCard);
        actions.put("3", this::handleRemoveCard);
        actions.put("4", this::handleRandomFlashCard);
        actions.put("5", this::handleInsertCardInBox);
        actions.put("6", this::handleMoveCardFromBox);
        actions.put("7", this::handleUpgradeCardFromBox);
    }
}
```

```
// O método assignActions agora pode ser usado para registrar as ações
public void assignActions() {
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
    actions.put("3", this::handleRemoveCard);
    actions.put("4", this::handleRandomFlashCard);
    actions.put("5", this::handleInsertCardInBox);
    actions.put("6", this::handleRemoveCardFromBox);
    actions.put("7", this::handleUpgradeCardFromBox);
    actions.put("8", this::handleDowngradeCardFromBox);
    actions.put("9", this::handleViewBoxes);
    actions.put("10", this::handleGetRandomCardFromBox); // Aqui você chama a

}

// Defina ou use seu Map de ações aqui (exemplo: actions)
private Map<String, Runnable> actions = new HashMap<>();

// Outras funções do controlador podem ser implementadas aqui.
}
```

Explicação:

Exemplo de uso no programa principal:

Se você precisar usar o controlador no seu código, pode criar e configurar o

`StudyCardsController` assim:

```
java Copiar código

public class Main {
    public static void main(String[] args) {
        LeitnerSystem leitnerSystem = new LeitnerSystem("Leitner Method");
        StudyCardsController controller = new StudyCardsController(leitnerSystem);
        controller.assignActions();

        // Agora você pode chamar a ação correspondente
        controller.handleGetRandomCardFromBox();
    }
}
```

Em resumo:

- O método `handleGetRandomCardFromBox` deve ser colocado dentro de uma classe de

`LeitnerSystem` para pegar e exibir um cartão aleatório.



```
void assignActions(){
    actions.put("1", this::handleViewCards);
    actions.put("2", this::handleCreateCard);
    actions.put("3", this::handleRemoveCard);
    actions.put("4", this::handleRandomFlashCard);
    actions.put("5", this::handleInsertCardInBox);
    actions.put("6", this::handleRemoveCardFromBox);
    actions.put("7", this::handleUpgradeCardFromBox);
    actions.put("8", this::handleDowngradeCardFromBox);
    actions.put("9", this::handleViewBoxes);
    actions.put("10", this::handleGetRandomCardFromBox); // Aqui
    agora está correto
}
```

```
java: invalid method reference
cannot find symbol
  symbol:   method handleRemoveCardFromBox()
  location: class org.example.controllers.StudyCardsController
```



O erro `cannot find symbol` geralmente ocorre quando o método referenciado não está presente ou não está acessível no contexto atual. No seu caso, a mensagem indica que o método `handleRemoveCardFromBox()` não foi encontrado dentro da classe `StudyCardsController`.

Para corrigir isso, verifique as seguintes possibilidades:

1. Verifique se o método `handleRemoveCardFromBox` está implementado na classe `StudyCardsController` ↓

1. Verifique se o método `handleRemoveCardFromBox` está implementado na classe `StudyCardsController`

Você precisa garantir que o método `handleRemoveCardFromBox` está realmente presente na classe e com a assinatura correta. Por exemplo:

java

 Copiar código

```
public class StudyCardsController {

    private LeitnerSystem leitnerSystem;

    public StudyCardsController(LeitnerSystem leitnerSystem) {
        this.leitnerSystem = leitnerSystem;
    }

    // Outros métodos de ação...

    // Certifique-se de que este método existe
    public void handleRemoveCardFromBox() {
        try {
            // Lógica para remover um cartão de uma caixa
            // Exemplo fictício: leitnerSystem.removeCardFromBox(cardId, boxId);
            System.out.println("Removing card from box...");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    // Outros métodos de ação continuam aqui...
}
```

```
public class StudyCardsController {

    private LeitnerSystem leitnerSystem;

    public StudyCardsController(LeitnerSystem leitnerSystem) {
        this.leitnerSystem = leitnerSystem;
    }

    void assignActions() {
        actions.put("1", this::handleViewCards);
        actions.put("2", this::handleCreateCard);
        actions.put("3", this::handleRemoveCard);
        actions.put("4", this::handleRandomFlashCard);
        actions.put("5", this::handleInsertCardInBox);
        actions.put("6", this::handleRemoveCardFromBox); // Método 'handleRemoveCar
        actions.put("7", this::handleUpgradeCardFromBox);
        actions.put("8", this::handleDowngradeCardFromBox);
        actions.put("9", this::handleViewBoxes);
        actions.put("10", this::handleGetRandomCardFromBox);
    }

    // Exemplo de um método de ação:
    public void handleRemoveCardFromBox() {
        try {
            // Lógica para remover um cartão da caixa
            System.out.println("Card removed from box.");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```



```

public String getRandomCardFromBox() {
    try {
        String response = this.leitnerSystem.getRandomCardFromBox(); // Aqui deve a
        return response;
    } catch (Exception e) {
        return e.getMessage(); // Retornar uma mensagem de erro, se necessário
    }
}

```

2. Verifique se `studyCardsController` foi instanciado corretamente

A variável `studyCardsController` deve ser uma instância válida da classe `StudyCardsController` no seu teste. Por exemplo, no seu código de teste, antes de chamar o método `getRandomCardFromBox()`, verifique se a instância de `studyCardsController` foi criada corretamente:

```

java Copiar código

@ExtendWith(MockitoExtension.class) // Se estiver usando Mockito
class YourTestClass {

    @Mock
    private LeitnerSystem leitnerSystem;

    @InjectMocks
    private StudyCardsController studyCardsController; // Certifique-se de que est

    @Test
    @Order(1)
    @DisplayName("Get Random Card From Box Test")
    void getRandomCardFromBox() {
        // Defina o comportamento esperado para o mock, se necessário
    }
}

```

corretamente:

java

 Copiar código

```
@ExtendWith(MockitoExtension.class) // Se estiver usando Mockito
class YourTestClass {

    @Mock
    private LeitnerSystem leitnerSystem;

    @InjectMocks
    private StudyCardsController studyCardsController; // Certifique-se de que est

    @Test
    @Order(1)
    @DisplayName("Get Random Card From Box Test")
    void getRandomCardFromBox() {
        // Defina o comportamento esperado para o mock, se necessário
        Mockito.when(leitnerSystem.getRandomCardFromBox()).thenReturn("Test Leitner

        String response = studyCardsController.getRandomCardFromBox();
        if (response == null) {
            fail("Response should not be null");
        }

        assertTrue(response.contains("Test LeitnerSystem"));

        if (response.contains("Test Random Leitner Question 1")) {
            assertTrue(response.contains("Test Random Leitner Answer 1"));
        } else {
            assertTrue(response.contains("Test Random Leitner Answer 2"));
        }
    }
}
```

```

        try {
            // Lógica para remover um cartão da caixa
            System.out.println("Card removed from box.");
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }

    // Outros métodos de ação, como handleViewCards, handleCreateCard, etc.
    public void handleViewCards() {
        System.out.println("Viewing cards...");
    }

    public void handleCreateCard() {
        System.out.println("Creating a card...");
    }

    // Outros métodos...
}

```

Resumo:

1. Verifique se o método `handleRemoveCardFromBox` está implementado corretamente na classe `StudyCardsController`.
2. Certifique-se de que todos os outros métodos de ação também estão implementados.
3. Garanta que os métodos tenham a visibilidade adequada.
4. Certifique-se de que a assinatura do método `handleRemoveCardFromBox` e de outros métodos correspondem às chamadas em `assignActions()`.

Se o erro persistir após essas verificações, compartilhe a implementação da classe para que eu

LONG METHOD

```
public String kanbanView() throws Exception {
    try{

        if(kanban.isEmpty()){
            throw new Exception("No material found");
        }
        StringBuilder sb = new StringBuilder();
        sb.append("[ Material ToDo: ");
        sb.append(System.lineSeparator());

        if(kanban.get(State.TODO).isEmpty()){
            sb.append("No material found");
        } else {
            for(PlannerMaterial material : kanban.get(State.TODO)){
                sb.append(", ").append(material.toString());
            }
        }
        sb.append(System.lineSeparator());
        sb.append("Material in progress:");
        sb.append(System.lineSeparator());
        if(kanban.get(State.DOING).isEmpty()){
            sb.append("No material found");
        } else {
            for(PlannerMaterial material : kanban.get(State.DOING)){
                sb.append(", ").append(material.toString());
            }
        }
        sb.append(System.lineSeparator());
        sb.append("Material completed:");
        sb.append(System.lineSeparator());
        if(kanban.get(State.DONE).isEmpty()){
```

```
            sb.append("Material in progress:");
            sb.append(System.lineSeparator());
            if(kanban.get(State.DOING).isEmpty()){
                sb.append("No material found");
            } else {
                for(PlannerMaterial material : kanban.get(State.DOING)){
                    sb.append(", ").append(material.toString());
                }
            }
            sb.append(System.lineSeparator());
            sb.append("Material completed:");
            sb.append(System.lineSeparator());
            if(kanban.get(State.DONE).isEmpty()){
                sb.append("No material found");
            } else {
                for(PlannerMaterial material : kanban.get(State.DONE)){
                    sb.append(", ").append(material.toString());
                }
            }
            sb.append("]");
            return sb.toString();
        } catch (Exception e){
            throw new Exception(e.getMessage());
        }
    }
}
```

refatore o code smell long method code utilizando a técnica de refatoração extract method



Kanban View Refactor





</> Kanban View Refactor

O método foi refatorado usando a técnica de *Extract Method*, extraindo a lógica de repetição para um novo método chamado `appendMaterials`. Isso melhora a legibilidade e reduz a duplicação de código.

```

1  public String kanbanView() throws Exception {
2  ~    try {
3  ~        if (kanban.isEmpty()) {
4              throw new Exception("No material found");
5          }
6
7          return buildKanbanView();
8  ~    } catch (Exception e) {
9          throw new Exception(e.getMessage());
10     }
11 }
12
13 ~ private String buildKanbanView() {
14     StringBuilder sb = new StringBuilder();
15     sb.append("[ Material ToDo: ").append(System.lineSeparator());
16     appendMaterials(sb, State.TODO, "No material found");
17
18     sb.append("Material in progress:").append(System.lineSeparator());
19     appendMaterials(sb, State.DOING, "No material found");
20
21     sb.append("Material completed:").append(System.lineSeparator());
22     appendMaterials(sb, State.DONE, "No material found");
23
24     sb.append("]");
25     return sb.toString();
26 }
27
28 ~ private void appendMaterials(StringBuilder sb, State state, String emptyMessage) {
29 ~     if (kanban.get(state).isEmpty()) {
30         sb.append(emptyMessage);
31 ~     } else {
32 ~         for (PlannerMaterial material : kanban.get(state)) {
33             sb.append(", ").append(material.toString());
34         }
35     }
36     sb.append(System.lineSeparator());
37 }
38

```

```

public String kanbanView() throws Exception {
    try {
        if (kanban.isEmpty()) {
            throw new Exception("No material found");
        }

        StringBuilder sb = new StringBuilder();
        sb.append("[ Material ToDo: ").append(System.lineSeparator());
        appendMaterials(sb, State.TODO, "No material found");

        sb.append("Material in
progress:").append(System.lineSeparator());
        appendMaterials(sb, State.DOING, "No material found");

        sb.append("Material
completed:").append(System.lineSeparator());
        appendMaterials(sb, State.DONE, "No material found");

        sb.append("]");
        return sb.toString();
    } catch (Exception e) {
        throw new Exception(e.getMessage());
    }
}

```

refatore o code smell long method code utilizando a técnica de refatoração extract method



</> Kanban View Refactor

```

25     return sb.toString();
26 }
27
28 private void appendMaterials(StringBuilder sb, State state, String emptyMessage) {
29     if (kanban.get(state).isEmpty()) {
30         sb.append(emptyMessage);
31     } else {
32         for (PlannerMaterial material : kanban.get(state)) {
33             sb.append(", ").append(material.toString());
34         }
35     }
36     sb.append(System.lineSeparator());
37 }
38

```

Editado

O método foi refatorado ainda mais para separar a construção da visualização em um método auxiliar chamado `buildKanbanView`. Isso reduz o comprimento do método principal, melhorando sua clareza e responsabilidade única.


```

1  public String kanbanView() throws Exception {
2  ✓    try {
3  ✓        if (kanban.isEmpty()) {
4            throw new Exception("No material found");
5        }
6
7        return buildKanbanView();
8  ✓    } catch (Exception e) {
9        throw new Exception(e.getMessage());
10    }
11 }
12
13 ✓ private String buildKanbanView() {
14     StringBuilder sb = new StringBuilder();
15     sb.append("[ Material ToDo: ").append(System.lineSeparator());
16     appendMaterials(sb, State.TODO, "No material found");
17
18     sb.append("Material in progress:").append(System.lineSeparator());
19     appendMaterials(sb, State.DOING, "No material found");
20
21     sb.append("Material completed:").append(System.lineSeparator());
22     appendMaterials(sb, State.DONE, "No material found");
23
24     sb.append("]");
25     return sb.toString();
26 }
27
28 ✓ private void appendMaterials(StringBuilder sb, State state, String emptyMessage) {
29 ✓     if (kanban.get(state).isEmpty()) {
30         sb.append(emptyMessage);
31 ✓     } else {
32 ✓         for (PlannerMaterial material : kanban.get(state)) {
33             sb.append(", ").append(material.toString());
34         }
35     }
36     sb.append(System.lineSeparator());
37 }
38

```

```

public String toString() {
    StringBuilder str = new StringBuilder();
    for (ToDo toDo : todos) {
        String toDoInfo = toDo.toString();
        str.append(toDoInfo);
        str.append("\n");
        Integer id = toDo.getId();
        List<LocalDateTime> todosDate = this.tracker.get(id);
        if (todosDate == null) {
            str.append("No tracks found\n");
        } else {
            for (LocalDateTime ldt : todosDate) {
                String pattern = "yyyy-MM-dd HH:mm:ss";
                DateTimeFormatter formatter =
                    DateTimeFormatter.ofPattern(pattern);
                String formattedDate = formatter.format(ldt);
                str.append(formattedDate);
                str.append("\n");
            }
        }
    }
    String response = str.toString();
    if (response.isEmpty()) {
        return "No ToDos found";
    }
    return response;
}

```

refatore o code smell long method code utilizando a técnica de refatoração extrair método

```

1  ~ public String toString() {
2      ~   StringBuilder str = new StringBuilder();
3
4  ~   for (ToDo toDo : todos) {
5       ~       appendToDoDetails(str, toDo);
6   ~   }
7
8       ~   String response = str.toString();
9       ~   return response.isEmpty() ? "No ToDos found" : response;
10  ~ }
11
12 ~ private void appendToDoDetails(StringBuilder str, ToDo toDo) {
13     ~   str.append(toDo.toString()).append("\n");
14     ~   appendTrackingDetails(str, toDo);

```

Responder no chat >

Mudanças realizadas:

1. Extração do método `appendToDoDetails`:
 - Responsável por adicionar as informações de um único `ToDo` no `StringBuilder`.
2. Extração do método `appendTrackingDetails`:
 - Responsável por adicionar os detalhes de rastreamento associados a um `ToDo`.
3. Extração do método `formatDate`:
 - Formatação de um `LocalDateTime` em uma string no formato desejado.

Essa refatoração torna o método principal mais legível e facilita o entendimento e a manutenção do código. Cada método agora tem uma única responsabilidade.

✕ Refactor Long Method

```
1  public String toString() {
2      StringBuilder str = new StringBuilder();
3
4      for (ToDo toDo : todos) {
5          appendToDoDetails(str, toDo);
6      }
7
8      String response = str.toString();
9      return response.isEmpty() ? "No Todos found" : response;
10 }
11
12 private void appendToDoDetails(StringBuilder str, ToDo toDo) {
13     str.append(toDo.toString()).append("\n");
14     appendTrackingDetails(str, toDo);
15 }
16
17 private void appendTrackingDetails(StringBuilder str, ToDo toDo) {
18     Integer id = toDo.getId();
19     List<LocalDateTime> todosDate = this.tracker.get(id);
20
21     if (todosDate == null) {
22         str.append("No tracks found\n");
23     } else {
24         for (LocalDateTime ldt : todosDate) {
25             str.append(formatDate(ldt)).append("\n");
26         }
27     }
28 }
29
30 private String formatDate(LocalDateTime dateTime) {
31     String pattern = "yyyy-MM-dd HH:mm:ss";
32     DateTimeFormatter formatter = DateTimeFormatter.ofPattern(pattern);
33     return formatter.format(dateTime);
34 }
35
```

```

public String setGoalSummary(){
    StringBuilder summary = new StringBuilder();
    summary.append("Goal Summary:\n").append("\n\n");
    if(this.isActive){
        summary.append("Active Goal:
\n").append(goal).append("\n\n");
    }
    if(this.isCompleted){
        summary.append("Completed Goal:
\n").append(goal).append("\n\n");
    }
    if(this.goalRequirements != null){
        summary.append("Requirements:\n");
        for(String requirement : this.goalRequirements){
            summary.append(requirement).append(", ");
        }
    }
    if(this.studyPlan != null){
        summary.append("Plan:\n");
        summary.append(this.studyPlan.toString());
    }
    if(this.studyObjective != null){
        summary.append("Objective:\n");
        summary.append(this.studyObjective.toString());
    }
    this.summary = summary.toString();
    return summary.toString();
}

```

refatore o código  long method code utilizando a técnica de refatoração extract method

java

 Copiar código

```
public String setGoalSummary() {
    StringBuilder summary = new StringBuilder();
    summary.append("Goal Summary:\n\n");

    appendActiveGoal(summary);
    appendCompletedGoal(summary);
    appendGoalRequirements(summary);
    appendStudyPlan(summary);
    appendStudyObjective(summary);

    this.summary = summary.toString();
    return summary.toString();
}

private void appendActiveGoal(StringBuilder summary) {
    if (this.isActive) {
        summary.append("Active Goal:\n").append(goal).append("\n\n");
    }
}

private void appendCompletedGoal(StringBuilder summary) {
    if (this.isCompleted) {
        summary.append("Completed Goal:\n").append(goal).append("\n\n");
    }
}

private void appendGoalRequirements(StringBuilder summary) {
    if (this.goalRequirements != null) {
        summary.append("Requirements: ↓ ");
        for (String requirement : this.goalRequirements) {
```

```
            for (String requirement : this.goalRequirements) {
                summary.append(requirement).append(", ");
            }
            summary.append("\n"); // Remove a vírgula final e insira uma quebra de linha
        }
    }

    private void appendStudyPlan(StringBuilder summary) {
        if (this.studyPlan != null) {
            summary.append("Plan:\n").append(this.studyPlan.toString()).append("\n");
        }
    }

    private void appendStudyObjective(StringBuilder summary) {
        if (this.studyObjective != null) {
            summary.append("Objective:\n").append(this.studyObjective.toString()).append("\n");
        }
    }
}
```

O que mudou?

1. **Extração de Métodos:** Dividimos as responsabilidades do método principal em vários métodos menores, cada um com uma única responsabilidade.
 - Métodos: `appendActiveGoal`, `appendCompletedGoal`, `appendGoalRequirements`, `appendStudyPlan` e `appendStudyObjective`.
2. **Legibilidade Melhorada:** Cada método agora tem um nome que reflete sua funcionalidade, facilitando o entendimento do código.
3. **Manutenção Facilitada:** Caso seja necessário modificar apenas um trecho do método, você pode focar no método específico sem impactar o restante.
4. **Evitar Repetição:** Se alguma funcionalidade precisar ser reutilizada em outro lugar, esses métodos menores podem ser chamados independentemente.

```
public Map<String, Integer> getReferenceCountMap(){
    Map<String, Integer> response = new HashMap<>();
    response.put("Audio References", 0);
    response.put("Video References", 0);
    response.put("Text References", 0);
    for (Reference reference : references) {
        if (reference.getClass() == AudioReference.class) {
            Integer audioCount = response.get("Audio References");
            response.put("Audio References", audioCount + 1);
        } else if (reference.getClass() == VideoReference.class) {
            if(((VideoReference) reference).handleStreamAvailability()){
                Integer videoCount = response.get("Video References");
                response.put("Video References", videoCount + 1);
            }
        } else if (reference.getClass() == TextReference.class){
            if(((TextReference) reference).handleTextAccess()){
                Integer textCount = response.get("Text References");
                response.put("Text References", textCount + 1);
            }
        }
    }
    setReferenceCount(response);
    return response;
}
```

refatore o code smell long method code utilizando a técnica de refatoração extract method

java

 Copiar código

```
public Map<String, Integer> getReferenceCountMap() {
    Map<String, Integer> response = initializeReferenceCountMap();

    for (Reference reference : references) {
        processReference(reference, response);
    }

    setReferenceCount(response);
    return response;
}

private Map<String, Integer> initializeReferenceCountMap() {
    Map<String, Integer> map = new HashMap<>();
    map.put("Audio References", 0);
    map.put("Video References", 0);
    map.put("Text References", 0);
    return map;
}

private void processReference(Reference reference, Map<String, Integer> response) {
    if (reference instanceof AudioReference) {
        incrementCount(response, "Audio References");
    } else if (reference instanceof VideoReference) {
        processVideoReference((VideoReference) reference, response);
    } else if (reference instanceof TextReference) {
        processTextReference((TextReference) reference, response);
    }
}
```



```
private void processVideoReference(VideoReference videoReference, Map<String, Integer> response) {
    if (videoReference.handleStreamAvailability()) {
        incrementCount(response, "Video References");
    }
}

private void processTextReference(TextReference textReference, Map<String, Integer> response) {
    if (textReference.handleTextAccess()) {
        incrementCount(response, "Text References");
    }
}

private void incrementCount(Map<String, Integer> response, String key) {
    response.put(key, response.get(key) + 1);
}
```

Alterações realizadas:

1. Extração de Métodos:

- Criação do método `initializeReferenceCountMap` para inicializar o mapa.
- Criação do método `processReference` para encapsular o processamento de cada referência.
- Criação de métodos específicos `processVideoReference` e `processTextReference` para lidar com cada tipo de referência.
- Criação do método utilitário `incrementCount` para incrementar os valores do mapa, reduzindo repetição.

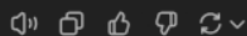
2. Substituição de verificações com `getClass` por `instanceof`:

- Utilizar `instanceof` melhora a legibilidade e permite uma abordagem mais flexível.

3. Legibilidade e modularidade:

- O método `getReferenceCountMap` agora está mais curto e delega responsabilidades específicas a métodos auxiliares.

Essa refatoração melhora a **legibilidade**, **manutenibilidade** e facilita futuras alterações no código.





```
void habitEmptyView() throws Exception {  
    try{  
        String response = kanbanView.kanbanView();  
        List<String> splitResponse=  
List.of(response.split(System.lineSeparator()));  
        int count = 0;  
        for(String str : splitResponse){  
            if(str.contains("No material found")){  
                count++;  
            }  
        }  
        assertEquals(3, count);  
        assertTrue(response.contains("No material found"));  
    } catch (Exception e){  
        fail();  
    }  
}
```

refatore o code smell long method code utilizando a técnica de refatoração extract method



Segue uma versão refatorada do código, utilizando a técnica de extração de métodos para lidar com o code smell "método longo". O objetivo é dividir a lógica em métodos menores, mais focados e reutilizáveis. Isso melhora a legibilidade e a manutenção do código.

java

 Copiar código

```
void habitEmptyView() throws Exception {
    try {
        String response = getKanbanResponse();
        List<String> splitResponse = splitResponseLines(response);
        int count = countNoMaterialFound(splitResponse);
        validateResponse(response, count);
    } catch (Exception e) {
        fail();
    }
}

private String getKanbanResponse() {
    return kanbanView.kanbanView();
}

private List<String> splitResponseLines(String response) {
    return List.of(response.split(System.lineSeparator()));
}

private int countNoMaterialFound(List<String> lines) {
    int count = 0;
    for (String line : lines) {
        if (line.contains("No material found")) {
            count++;
        }
    }
    return count;
}
```



```
private void validateResponse(String response, int count) {
    assertEquals(3, count);
    assertTrue(response.contains("No material found"));
}
```

Alterações realizadas:

1. Extração de métodos menores:

- `getKanbanResponse()` : Obtém a resposta do `kanbanView`.
- `splitResponseLines(String response)` : Divide a resposta em linhas.
- `countNoMaterialFound(List<String> lines)` : Conta as ocorrências da string "No material found".
- `validateResponse(String response, int count)` : Valida a resposta e o contador.

2. Melhoria da legibilidade:

Cada método possui uma única responsabilidade e nomes que descrevem claramente o propósito.

3. Manutenção facilitada:

Alterações futuras em qualquer lógica podem ser feitas em métodos específicos, sem afetar a estrutura geral do código.

Agora, o método principal (`habitEmptyView`) é mais limpo e compreensível, enquanto a lógica foi delegada a métodos menores.

java

Copiar código

```
public Todo(Integer id, String title, String description, int priority) {
    validateId(id);
    validateTitle(title);
    validateDescription(description);

    this.id = id;
    this.title = title;
    this.description = description;
    setPriority(priority);
    this.completed = false;
}

private void validateId(Integer id) {
    if (id == null || id <= 0) {
        throw new IllegalArgumentException("ID must be a positive integer.");
    }
}

private void validateTitle(String title) {
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("Title cannot be null or blank.");
    }
}

private void validateDescription(String description) {
    if (description == null || description.isBlank()) {
        throw new IllegalArgumentException("Description cannot be null or blank.");
    }
}
```



Envie uma mensagem para o ChatGPT



O ChatGPT pode cometer erros. Considere verificar informações importantes.



```
public ToDo(Integer id, String title, String description, int priority) {  
    if (id == null || id <= 0) {  
        throw new IllegalArgumentException("ID must be a positive  
integer.");  
    }  
    if (title == null || title.isBlank()) {  
        throw new IllegalArgumentException("Title cannot be null or  
blank.");  
    }  
    if (description == null || description.isBlank()) {  
        throw new IllegalArgumentException("Description cannot be  
null or blank.");  
    }  
    this.id = id;  
    this.title = title;  
    this.description = description;  
    setPriority(priority);  
    this.completed = false;  
}
```

refatore o code smell long method code utilizando a técnica de refatoração extract method



A técnica de **"Extract Method"** pode ser usada para melhorar a legibilidade e a manutenção do código, movendo partes específicas para métodos auxiliares. Nesse caso, podemos extrair as validações para métodos separados, reduzindo o tamanho e a complexidade do construtor.

Aqui está o código refatorado:

java



Copiar código

Envie uma mensagem para o ChatGPT



O ChatGPT pode cometer erros. Considere verificar informações importantes.

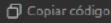
DATA CLASS



```
public class Card {  
    private String question;  
    private String answer;  
  
    public Card(String question, String answer) {  
        this.question = question;  
        this.answer = answer;  
    }  
  
    public String getQuestion() {  
        return question;  
    }  
  
    public void setQuestion(String question) {  
        this.question = question;  
    }  
  
    public String getAnswer() {  
        return answer;  
    }  
  
    public void setAnswer(String answer) {  
        this.answer = answer;  
    }  
  
    public void edit(String question, String answer) {  
        setQuestion(question);  
        setAnswer(answer);  
    }  
}
```



```
public class Card {  
    private String question;  
    private String answer;  
  
    public Card(String question, String answer) {  
        if (question == null || question.isBlank()) {  
            throw new IllegalArgumentException("Question cannot be null or blank");  
        }  
        if (answer == null || answer.isBlank()) {  
            throw new IllegalArgumentException("Answer cannot be null or blank");  
        }  
        this.question = question;  
        this.answer = answer;  
    }  
  
    public String displayCard() {  
        return "Q: " + question + "\nA: " + answer;  
    }  
  
    public boolean isCorrectAnswer(String providedAnswer) {  
        return answer.equalsIgnoreCase(providedAnswer.trim());  
    }  
  
    public void update(String newQuestion, String newAnswer) {  
        if (newQuestion != null && !newQuestion.isBlank()) {  
            this.question = newQuestion;  
        }  
        if (newAnswer != null && !newAnswer.isBlank()) {  
            this.answer = newAnswer;  
        }  
    }  
}
```



Alterações realizadas:

1. **Adicionado o método `displayCard`:**
 - Responsável por formatar a exibição do cartão com a pergunta e a resposta.
2. **Adicionado o método `isCorrectAnswer`:**
 - Verifica se uma resposta fornecida pelo usuário é correta, adicionando lógica significativa à classe.
3. **Refatorado o método `update`:**
 - Substitui o setter direto para evitar alterações externas descontroladas e garante que apenas valores válidos sejam atualizados.
4. **Removidos os setters públicos:**
 - Agora os atributos só podem ser alterados através de métodos com lógica específica.
5. **Validações nos campos no construtor:**
 - Garante que a pergunta e a resposta não sejam nulas ou em branco, protegendo a integridade do objeto.

Benefícios:

- **Encapsulamento:** A classe agora protege melhor seus dados e expõe apenas métodos necessários.
- **Cohesão:** A lógica está encapsulada dentro da classe, reduzindo a necessidade de manipulação externa.
- **Evita inconsistências:** As validações garantem que os dados do objeto estejam sempre em um estado válido.





```
public abstract class Reference {  
    private String title;  
    private String description;  
    private String link;  
    private String accessRights;  
    private String license;  
    private boolean isDownloadable;  
    private int rating;  
    private String language;  
    private int viewCount;  
    private int downloadCount;  
    private int shareCount;  
  
    public void setTitle(String title) {  
        this.title = title;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public void setDescription(String description) {  
        this.description = description;  
    }  
  
    public String getDescription() {  
        return description;  
    }  
  
    public void set↓ (String link) {  
        this.link = link;  
    }  
}
```

Envie uma mensagem para o ChatGPT

```
        this.description = description;
    }

    public String getDescription() {
        return description;
    }

    public void setLink(String link) {
        this.link = link;
    }

    public String getLink() {
        return link;
    }

    public String getAccessRights() {
        return accessRights;
    }

    public void setAccessRights(String accessRights) {
        this.accessRights = accessRights;
    }

    public String getLicense() {
        return license;
    }

    public void setLicense(String license) {
        this.license = license;
    }

    public boolean .IsDownloadable() {
        return isDownloadable;
    }
}
```

Envie uma mensagem para o ChatGPT

```
}

public boolean getIsDownloadable() {
    return isDownloadable;
}

public void setDownloadable(boolean downloadable) {
    isDownloadable = downloadable;
}

public int getRating() {
    return rating;
}

public void setRating(int rating) {
    this.rating = rating;
}

public String getLanguage() {
    return language;
}

public void setLanguage(String language) {
    this.language = language;
}

public int getViewCount() {
    return viewCount;
}

public void setViewCount(int viewCount) {
    this.viewCount = viewCount;
}
```

Envie uma mensagem para o ChatGPT

```
        this.language = language;
    }

    public int getViewCount() {
        return viewCount;
    }

    public void setViewCount(int viewCount) {
        this.viewCount = viewCount;
    }

    public int getDownloadCount() {
        return downloadCount;
    }

    public void setDownloadCount(int downloadCount) {
        this.downloadCount = downloadCount;
    }

    public int getShareCount() {
        return shareCount;
    }

    public void setShareCount(int shareCount) {
        this.shareCount = shareCount;
    }
}
```

refatore o code smell Data Class code utilizando a técnica de refatoração



Envie uma mensagem para o ChatGPT

Passos de Refatoração:

1. **Identificar Lógica Relevante:** Determine comportamentos que podem ser movidos para a classe.
2. **Encapsular Operações:** Adicione métodos que processam ou validam os dados em vez de expô-los diretamente.
3. **Reduzir Getters/Setters Simples:** Transforme métodos que apenas retornam ou definem valores em métodos mais significativos.

Aqui está a versão refatorada:

 Refactor Data Class 

```
1  public abstract class Reference {  
2      private String title;  
3      private String description;  
4      private String link;  
5      private String accessRights;  
6      private String license;  
7      private boolean isDownloadable;  
8      private int rating;  
9      private String language;  
10     private int viewCount;  
11     private int downloadCount;  
12     private int shareCount;  
13  
14     // Constructor to initialize mandatory fields
```

Responder no chat >

Melhorias realizadas:

1. **Validação nos Métodos de Atualização.**  Campos como `title`, `link`, e `language` agora têm

Envie uma mensagem para o ChatGPT

```

v public abstract class Reference {
    private String title;
    private String description;
    private String link;
    private String accessRights;
    private String license;
    private boolean isDownloadable;
    private int rating;
    private String language;
    private int viewCount;
    private int downloadCount;
    private int shareCount;

    // Constructor to initialize mandatory fields
v public Reference(String title, String link, String language) {
v     if (title == null || title.isEmpty() || link == null || link.isEmpty() || language == null || language.isEmpty()) {
        throw new IllegalArgumentException("Title, link, and language cannot be null or empty.");
    }
    this.title = title;
    this.link = link;
    this.language = language;
}

    // Getters and setters with encapsulated logic where applicable

v public String getTitle() {
    return title;
}

v public void updateTitle(String title) {
v     if (title == null || title.isEmpty()) {
        throw new IllegalArgumentException("Title cannot be null or empty.");
    }
    this.title = title;
}

v public String getDescription() {
    return description;
}

v public void updateDescription(String description) {

```

```

    }
}

public String getDescription() {
    return description;
}

public void updateDescription(String description) {
    this.description = description;
}

public String getLink() {
    return link;
}

public String getAccessRights() {
    return accessRights;
}

public void updateAccessRights(String accessRights) {
    if (!isValidAccessRights(accessRights)) {
        throw new IllegalArgumentException("Invalid access rights value.");
    }
    this.accessRights = accessRights;
}

public String getLicense() {
    return license;
}

public void updateLicense(String license) {
    this.license = license;
}

public boolean isDownloadable() {
    return isDownloadable;
}

public void toggleDownloadable() {
    this.isDownloadable = !this.isDownloadable;
}

public int getRating() {

```

```
✓ public int getRating() {  
    return rating;  
}  
  
✓ public void rate(int rating) {  
✓     if (rating < 1 || rating > 5) {  
         throw new IllegalArgumentException("Rating must be between 1 and 5.");  
     }  
     this.rating = rating;  
}  
  
✓ public String getLanguage() {  
    return language;  
}  
  
✓ public int getViewCount() {  
    return viewCount;  
}  
  
✓ public void incrementViewCount() {  
    this.viewCount++;  
}  
  
✓ public int getDownloadCount() {  
    return downloadCount;  
}  
  
✓ public void incrementDownloadCount() {  
✓     if (isDownloadable) {  
         this.downloadCount++;  
✓     } else {  
         throw new IllegalStateException("Resource is not downloadable.");  
     }  
}  
  
✓ public int getShareCount() {  
    return shareCount;  
}  
  
✓ public void incrementShareCount() {  
    this.shareCount++;  
}
```



```

    if (!isDownloadable) {
        this.downloadCount++;
    } else {
        throw new IllegalStateException("Resource is not downloadable.");
    }
}

public int getShareCount() {
    return shareCount;
}

public void incrementShareCount() {
    this.shareCount++;
}

// Example of encapsulated logic
private boolean isValidAccessRights(String accessRights) {
    return accessRights != null && (accessRights.equals("Public") || accessRights.equals("Private") || accessRights.equals("Protected"));
}

// High-level behavior
public String summarize() {
    return String.format("Title: %s, Language: %s, Views: %d, Downloads: %d", title, language, viewCount, downloadCount);
}

@Override
public String toString() {
    return String.format("Reference[Title: %s, Link: %s, Language: %s, Rating: %d]", title, link, language, rating);
}
}

```



```
public class ToDo implements PlannerMaterial{
    private Integer id;
    private String title;
    private String description;
    private int priority;

    public ToDo(Integer id, String title, String description, int priority) {
        this.id = id;
        this.title = title;
        this.description = description;
        this.priority = priority;
    }

    @Override
    public String toString() {
        return MessageFormat.format("[(Priority:{3}) ToDo {0}: {1}, {2}]",
id, title, description, priority);
    }

    public int getId() {
        return id;
    }

    public void setId(Integer id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }
}
```



Envie uma mensagem para o ChatGPT

```
public void setId(Integer id) {
    this.id = id;
}

public String getTitle() {
    return title;
}

public void setTitle(String title) {
    this.title = title;
}

public String getDescription() {
    return description;
}

public void setDescription(String description) {
    this.description = description;
}

public int getPriority() {
    return priority;
}

public void setPriority(int priority) {
    this.priority = priority;
}
}
```

refatore o code smell Data Class code utilizando a técnica de refatoração



Envie uma mensagem para o ChatGPT

```

import java.text.MessageFormat;

public class Todo implements PlannerMaterial {
    private Integer id;
    private String title;
    private String description;
    private int priority;
    private boolean completed;

    public Todo(Integer id, String title, String description, int priority) {
        if (id == null || id <= 0) {
            throw new IllegalArgumentException("ID must be a positive integer.");
        }
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title cannot be null or blank.");
        }
        if (description == null || description.isBlank()) {
            throw new IllegalArgumentException("Description cannot be null or blank.");
        }
        this.id = id;
        this.title = title;
        this.description = description;
        setPriority(priority);
        this.completed = false;
    }

    // Adds behavior to mark a task as completed
    public void markAsCompleted() {
        if (completed) {
            throw new IllegalStateException("Task is already completed.");
        }
        this.completed = true;
    }

    // Adds behavior to reset task status
    public void resetStatus() {
        this.completed = false;
    }

    // validates priority to ensure it is within a valid range
    public void setPriority(int priority) {

```

```

    }

    // Provides a more detailed summary of the task
    public String getSummary() {
        return MessageFormat.format(
            "Task ID: {0}\nTitle: {1}\nDescription: {2}\nPriority: {3}\nStatus: {4}",
            id, title, description, priority, completed ? "Completed" : "Pending"
        );
    }

    // Compares tasks based on priority (higher priority first)
    public int compareTo(Todo other) {
        return Integer.compare(other.priority, this.priority);
    }

    @Override
    public String toString() {
        return MessageFormat.format(
            "[({Priority:{3}}) Todo {0}: {1}, {2}, Status: {4}]",
            id, title, description, priority, completed ? "Completed" : "Pending"
        );
    }

    // Getter and setter methods

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        if (id == null || id <= 0) {
            throw new IllegalArgumentException("ID must be a positive integer.");
        }
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title cannot be null or blank.");
        }
        this.title = title;
    }
}

```

```

✓ public void setId(Integer id) {
✓     if (id == null || id <= 0) {
        throw new IllegalArgumentException("ID must be a positive integer.");
    }
    this.id = id;
}

✓ public String getTitle() {
    return title;
}

✓ public void setTitle(String title) {
✓     if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("Title cannot be null or blank.");
    }
    this.title = title;
}

✓ public String getDescription() {
    return description;
}

✓ public void setDescription(String description) {
✓     if (description == null || description.isBlank()) {
        throw new IllegalArgumentException("Description cannot be null or blank.");
    }
    this.description = description;
}

✓ public int getPriority() {
    return priority;
}

✓ public boolean isCompleted() {
    return completed;
}
}

```

```

import java.text.MessageFormat;

public class Todo implements PlannerMaterial {
    private Integer id;
    private String title;
    private String description;
    private int priority;
    private boolean completed;

    public Todo(Integer id, String title, String description, int priority) {
        if (id == null || id <= 0) {
            throw new IllegalArgumentException("ID must be a positive integer.");
        }
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title cannot be null or blank.");
        }
        if (description == null || description.isBlank()) {
            throw new IllegalArgumentException("Description cannot be null or blank.");
        }
        this.id = id;
        this.title = title;
        this.description = description;
        setPriority(priority);
        this.completed = false;
    }

    // Adds behavior to mark a task as completed
    public void markAsCompleted() {
        if (completed) {
            throw new IllegalStateException("Task is already completed.");
        }
        this.completed = true;
    }

    // Adds behavior to reset task status
    public void resetStatus() {
        this.completed = false;
    }

    // Validates priority to ensure it is within a valid range
    public void setPriority(int priority) {
        if (priority < 1 || priority > 5) {
            throw new IllegalArgumentException("Priority must be between 1 and 5.");
        }
        this.priority = priority;
    }

```

```

    }

    // Provides a more detailed summary of the task
    public String getSummary() {
        return MessageFormat.format(
            "Task ID: {0}\nTitle: {1}\nDescription: {2}\nPriority: {3}\nStatus: {4}",
            id, title, description, priority, completed ? "Completed" : "Pending"
        );
    }

    // Compares tasks based on priority (higher priority first)
    public int compareTo(Todo other) {
        return Integer.compare(other.priority, this.priority);
    }

    @Override
    public String toString() {
        return MessageFormat.format(
            "[{0}]\nPriority: {1} Todo {2}: [{3}], [{4}], Status: {5}",
            id, title, description, priority, completed ? "Completed" : "Pending"
        );
    }

    // Getter and setter methods

    public Integer getId() {
        return id;
    }

    public void setId(Integer id) {
        if (id == null || id <= 0) {
            throw new IllegalArgumentException("ID must be a positive integer.");
        }
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title cannot be null or blank.");
        }
        this.title = title;
    }
}

```



```

~ public String getTitle() {
    return title;
}

~ public void setTitle(String title) {
~     if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("Title cannot be null or blank.");
    }
    this.title = title;
}

~ public String getDescription() {
    return description;
}

~ public void setDescription(String description) {
~     if (description == null || description.isBlank()) {
        throw new IllegalArgumentException("Description cannot be null or blank.");
    }
    this.description = description;
}

~ public int getPriority() {
    return priority;
}

~ public boolean isCompleted() {
    return completed;
}
}
```



```
public class Task extends Registry{
    private String title;
    private String description;
    private String author;
    private LocalDateTime date;

    public Task(String title, String description, String author,
LocalDateTime date) {
        this.title = title;
        this.name = title;
        this.description = description;
        this.author = author;
        this.date = date;
    }
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
        this.description = description;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String author) {
        this.author = author;
    }
}
```

Envie uma mensagem para o ChatGPT



O ChatGPT pode cometer erros. Considere verificar informações importantes.

```
public void setTitle(String title) {  
    this.title = title;  
}  
public String getDescription() {  
    return description;  
}  
public void setDescription(String description) {  
    this.description = description;  
}  
public String getAuthor() {  
    return author;  
}  
public void setAuthor(String author) {  
    this.author = author;  
}  
public LocalDateTime getDate() {  
    return date;  
}  
public void setDate(LocalDateTime date) {  
    this.date = date;  
}  
}
```

refatore o code smell Data Class code utilizando a técnica de refatoração



Para refatorar o code smell **Data Class** no código apresentado, podemos aplicar a técnica de refatoração adicionando comportamento significativo à classe. O objetivo é garantir que a classe não seja apenas um recipiente para armazenar dados, mas que também encapsule lógica relacionada aos dados que ela mantém.



Aqui está um exemplo de como o código pode ser refatorado para adicionar comportamento

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Task extends Registry {
    private String title;
    private String description;
    private String author;
    private LocalDateTime date;

    public Task(String title, String description, String author, LocalDateTime date) {
        this.title = title;
        this.name = title; // Supondo que 'name' é herdado de Registry
        this.description = description;
        this.author = author;
        this.date = date;
    }

    // Getters e Setters
    public String getTitle() {
        return title;
    }
    public void setTitle(String title) {
        this.title = title;
    }
    public String getDescription() {
        return description;
    }
    public void setDescription(String description) {
        this.description = description;
    }
    public String getAuthor() {
        return author;
    }
}
```

 Copiar código



```
        this.description = description;
    }
    public String getAuthor() {
        return author;
    }
    public void setAuthor(String author) {
        this.author = author;
    }
    public LocalDateTime getDate() {
        return date;
    }
    public void setDate(LocalDateTime date) {
        this.date = date;
    }

    // Adicionando comportamento significativo

    /**
     * Verifica se a tarefa está atrasada.
     *
     * @return true se a data da tarefa for anterior ao momento atual, caso contrário false.
     */
    public boolean isOverdue() {
        return date.isBefore(LocalDateTime.now());
    }

    /**
     * Formata a data da tarefa em um formato legível.
     *
     * @return Data formatada como String.
     */
    public String getFormattedDate() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    }
}
```

 Copiar código

```

    }

    /**
     * Formata a data da tarefa em um formato legível.
     *
     * @return Data formatada como String.
     */
    public String getFormattedDate() {
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
        return date.format(formatter);
    }

    /**
     * Exibe informações completas sobre a tarefa.
     *
     * @return Uma String com detalhes formatados da tarefa.
     */
    public String displayDetails() {
        return String.format("Task: %s\nDescription: %s\nAuthor: %s\nDate: %s\nStatus: %s\n",
            title,
            description,
            author,
            getFormattedDate(),
            isOverdue() ? "Overdue" : "On Schedule");
    }
}

```

 Copiar código

Melhorias Adicionadas:

1. Encapsulamento de Lógica de Negócio:

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class Task extends Registry {
    private final String title;
    private final String description;
    private final String author;
    private final LocalDateTime date;

    public Task(String title, String description, String author, LocalDateTime date) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title cannot be null or blank");
        }
        if (description == null || description.isBlank()) {
            throw new IllegalArgumentException("Description cannot be null or blank");
        }
        if (author == null || author.isBlank()) {
            throw new IllegalArgumentException("Author cannot be null or blank");
        }
        if (date == null) {
            throw new IllegalArgumentException("Date cannot be null");
        }

        this.title = title;
        this.name = title; // Supondo que 'name' é herdado de Registry
        this.description = description;
        this.author = author;
        this.date = date;
    }

    // Getters (removemos setters para garantir imutabilidade onde aplicável)
```

```
this.name = title; // Supondo que 'name' é herdado de Registrado
this.description = description;
this.author = author;
this.date = date;
}

// Getters (removemos setters para garantir imutabilidade onde aplicável)
public String getTitle() {
    return title;
}

public String getDescription() {
    return description;
}

public String getAuthor() {
    return author;
}

public LocalDateTime getDate() {
    return date;
}

// Adicionando comportamento significativo

/**
 * Verifica se a tarefa está atrasada.
 *
 * @return true se a data da tarefa for anterior ao momento atual, caso contrário false
 */
public boolean isOverdue() {
    return date.isBefore(LocalDate.now());
}
```



```
/**
 * Formata a data da tarefa em um formato legível.
 *
 * @return Data formatada como String.
 */
public String getFormattedDate() {
    DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm");
    return date.format(formatter);
}

/**
 * Retorna o status da tarefa.
 *
 * @return "Overdue" se a tarefa estiver atrasada, caso contrário, "On Schedule"
 */
public String calculateStatus() {
    return isOverdue() ? "Overdue" : "On Schedule";
}

/**
 * Representação textual da tarefa.
 *
 * @return Uma String com os detalhes da tarefa.
 */
@Override
public String toString() {
    return String.format("Task: %s\nDescription: %s\nAuthor: %s\nDate: %s\nStatus: %s",
        title, description, author, getFormattedDate(), calculateStatus());
}
}
```

1. Validação no Construtor:

- Garante que o objeto seja criado apenas com valores válidos.

2. Remoção de Setters:

- Para `title`, `description`, `author` e `date`, os setters foram removidos para preservar o estado inicial do objeto.

3. Método `calculateStatus`:

- Centraliza a lógica do status para facilitar futuras alterações e melhora a legibilidade.

4. Uso do Método `toString`:

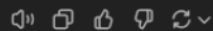
- Substituímos o método `displayDetails` por uma implementação de `toString`, uma prática padrão para representar objetos.

5. Código mais claro e coeso:

- Nomeação dos métodos e encapsulamento de lógica para melhor organização.

Benefícios:

- **Menos Propenso a Erros:** Validações e imutabilidade reduzem a possibilidade de inconsistências.
- **Fácil Extensibilidade:** Comportamentos encapsulados em métodos permitem evoluir a classe sem afetar o restante do código.
- **Mais Legível e Profissional:** Segue boas práticas que tornam o código mais claro e padronizado.



LONG PARAMETER LIST



```
public int addHabit(String name, String motivation, Integer
dailyMinutesDedication, Integer dailyHoursDedication, Integer year,
Integer month, Integer day, Integer hour, Integer minute, Integer
seconds, Boolean isConcluded) {
    LocalDateTime lt = LocalDateTime.of(dailyHoursDedication,
dailyMinutesDedication);
    LocalDateTime startDate = LocalDateTime.of(year, month, day,
hour, minute, seconds);
    Habit habit = new Habit(name, motivation, lt, this.nextId,
startDate, isConcluded);
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}
```

refatore Long Parament List code smell utilizando a técnica de refatoração move mehod

codigo completo

```
package org.example.studyplanner;

import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class HabitTracker {
```

```
package org.example.studyplanner;

import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.format.DateTimeFormatter;
import java.util.*;

public class HabitTracker {
    private List<Habit> habits;
    private Map<Integer, List<LocalDateTime>> tracker;
    private Integer nextId;

    private static HabitTracker instance;

    public static HabitTracker getHabitTracker() {
        if (instance == null) {
            instance = new HabitTracker();
        }
        return instance;
    }

    private HabitTracker(){
        this.habits = new ArrayList<>();
        this.tracker = new HashMap<>();
        this.nextId = 1;
    }

    public String viewAllHabitRecords() {
        StringBuilder response = new StringBuilder();
        for (Habit habit : habits) {
            response.append(formatHabitWithRecords(habit));
        }
    }
}
```

```

return response.toString();
}

private String formatHabitWithRecords(Habit habit) {
    StringBuilder habitRecord = new StringBuilder();
    habitRecord.append("[ Habit: ")
        .append(habit.getName())
        .append(". Records: ");

    List<LocalDateTime> records = tracker.get(habit.getId());
    habitRecord.append(formatHabitRecords(records));
    habitRecord.append("]");

    return habitRecord.toString();
}

private String formatHabitRecords(List<LocalDateTime> records) {
    StringBuilder formattedRecords = new StringBuilder();
    for (LocalDateTime record : records) {

formattedRecords.append(formatHabitDate(record)).append(", ");
    }
    if (!records.isEmpty()) {
        // Remove a última vírgula e espaço adicionados
        formattedRecords.setLength(formattedRecords.length() - 2);
    }
    return formattedRecords.toString();
}

@Override
public String toString() {
    StringBuilder response = new StringBuilder();
    for (Habit habit : habits) {

```

```

formattedRecords.append(formatHabitDate(record)).append(", ");
    }
    if (!records.isEmpty()) {
        // Remove a última vírgula e espaço adicionados
        formattedRecords.setLength(formattedRecords.length() - 2);
    }
    return formattedRecords.toString();
}

@Override
public String toString() {
    StringBuilder response = new StringBuilder();
    for (Habit habit : habits) {
        response.append(habit.toString()).append(", ");
    }
    return "Habits: " + response.toString();
}

public Habit getHabitById(Integer id){
    return this.habits.stream()
        .filter(habit -> Objects.equals(habit.getId(), id))
        .findFirst().orElse(null);
}

public List<Habit> getHabits() {
    return this.habits;
}

public String formatHabitDate(LocalDate date){
    DateTimeFormatter formatter =
    DateTimeFormal ↓ ofPattern("dd-MM-yyyy HH:mm:ss");

```

```

    public int addHabit(String name, String motivation, Integer
dailyMinutesDedication, Integer dailyHoursDedication, Integer year,
Integer month, Integer day, Integer hour, Integer minute, Integer
seconds, Boolean isConcluded) {
    LocalTime lt = LocalTime.of(dailyHoursDedication,
dailyMinutesDedication);
    LocalDateTime startDate = LocalDateTime.of(year, month, day,
hour, minute, seconds);
    Habit habit = new Habit(name, motivation, lt, this.nextId,
startDate, isConcluded);
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}

    public int handleAddHabitAdapter(List<String> stringProperties,
List<Integer> intProperties, boolean isConcluded){
    return addHabit(stringProperties.get(0), stringProperties.get(1),
intProperties.get(0), intProperties.get(1), intProperties.get(2),
intProperties.get(3), intProperties.get(4), intProperties.get(5),
intProperties.get(6), intProperties.get(7), isConcluded);
}

    public int addHabit(String name, String motivation) {

    Habit habit = new Habit(name, motivation, this.nextId);
    this.habits.add(habit);
    int response = nextId;

```

```
public void addHabitRecord(Integer id){
    tracker.get(id).add(LocalDateTime.now());
}

public void toggleConcludeHabit(Integer id) {
    for (Habit habit : this.habits) {
        if (habit.getId().equals(id)) {
            habit.setIsConcluded(!habit.getIsConcluded());
        }
    }
}

public void removeHabit(Integer id) {
    this.habits.removeIf(habit -> habit.getId().equals(id));
    this.tracker.remove(id);
}

public List<LocalDate> getHabitRecords(Integer id) {
    return this.tracker.get(id);
}

public List<String> searchInHabits(String search){
    List<String> habits = new ArrayList<>();
    for (Habit habit : this.habits) {
        if
(habit.getName().toLowerCase().contains(search.toLowerCase()) ||
habit.getMotivation().toLowerCase().contains(search.toLowerCase())
){
            habits.add(habit.toString());
        }
    }
    return habits;
}
```



```
import java.time.LocalDateTime;
import java.time.LocalTime;

public class Habit {
    private String name;
    private String motivation;
    private LocalTime dailyDedication;
    private Integer id;
    private LocalDateTime startDate;
    private Boolean isConcluded;

    // Construtor para simplificação no exemplo
    public Habit(String name, String motivation, Integer id) {
        this.name = name;
        this.motivation = motivation;
        this.id = id;
    }

    public Habit(String name, String motivation, LocalTime dailyDedication, Integer id, LocalDateTime startDate, Boolean isConcluded) {
        this.name = name;
        this.motivation = motivation;
        this.dailyDedication = dailyDedication;
        this.id = id;
        this.startDate = startDate;
        this.isConcluded = isConcluded;
    }

    public static Habit create(String name, String motivation, Integer dailyHoursDedication, Integer id, Integer year, Integer month, Integer day, Integer hour, Integer minute, Boolean isConcluded) {
        LocalTime dailyDedication = LocalTime.of(dailyHoursDedication, dailyMinutesDedication);
        LocalDateTime startDate = LocalDateTime.of(year, month, day, hour, minute, second);
        return new Habit(name, motivation, dailyDedication, id, startDate, isConcluded);
    }
}
```

 Copiar código

```
        this.isConcluded = isConcluded;
    }

    public static Habit create(String name, String motivation, Integer dailyMinutes,
                               Integer id, Integer year, Integer month, Integer day,
                               LocalTime dailyDedication = LocalTime.of(dailyHoursDedication, dailyMinutes),
                               LocalDateTime startDate = LocalDateTime.of(year, month, day, hour, minute),
                               Boolean isConcluded = false) {
        return new Habit(name, motivation, dailyDedication, id, startDate, isConcluded);
    }

    public String getName() {
        return name;
    }

    public Integer getId() {
        return id;
    }

    public void setIsConcluded(Boolean isConcluded) {
        this.isConcluded = isConcluded;
    }

    @Override
    public String toString() {
        return "Habit{" +
            "name='" + name + '\'' +
            ", motivation='" + motivation + '\'' +
            ", id=" + id +
            '}';
    }
}
```



 Copiar código

```

public String toString() {
    return "Habit{" +
        "name='" + name + '\'' +
        ", motivation='" + motivation + '\'' +
        ", id=" + id +
        '}';
}
}

```

 Copiar código

Classe `HabitTracker`

No método `addHabit`, utilizamos o método `Habit.create` para encapsular a criação do hábito:

```

java
public int addHabit(String name, String motivation, Integer dailyMinutesDedication,
    Integer year, Integer month, Integer day, Integer hour, Integer minute) {
    Habit habit = Habit.create(name, motivation, dailyMinutesDedication, dailyHours,
        dailyMinutes, year, month, day, hour, minute);
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}

```

 Copiar código

Benefícios

1. **Redução de Parâmetros:** A lógica de construção de `Habit` é movida para o método `Habit.create`, deixando o método `addHabit` mais limpo e fácil de entender.
2. **Encapsulamento:** A lógica para criar e inicializar um `Habit` fica na própria classe `Habit`, onde pertence.
3. **Reutilização:** Se outro método ou classe precisar criar um `Habit`, ele pode reutilizar o método



java

 Copiar código

```
package org.example.studyplanner;

import java.text.MessageFormat;
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.LocalTime;

public class Habit implements PlannerMaterial {
    private Integer id;
    private Boolean isConcluded;
    private String name;
    private String motivation;
    private LocalTime dailyDedicationTime;
    private LocalDateTime startDate;

    // Construtores existentes
    Habit(String name, String motivation, LocalTime dailyDedicationTime, Integer id, Boolean isConcluded) {
        this.isConcluded = isConcluded;
        this.id = id;
        this.name = name;
        this.motivation = motivation;
        this.dailyDedicationTime = dailyDedicationTime;
        this.startDate = startDate;
    }

    Habit(String name, String motivation, Integer id) {
        this.isConcluded = false;
        this.id = id;
        this.name = name;
        this.motivation = motivation;
    }
}
```

```

        this.isConcluded = false;
        this.id = id;
        this.name = name;
        this.motivation = motivation;
    }

    // Método estático para criar Habit de maneira mais simples
    public static Habit create(String name, String motivation, Integer dailyMinutes,
                               Integer id, Integer year, Integer month, Integer day,
                               LocalTime dailyDedication = LocalTime.of(dailyHoursDedication, dailyMinutes),
                               LocalDateTime startDate = LocalDateTime.of(year, month, day, hour, minute),
                               Boolean isConcluded = false) {
        return new Habit(name, motivation, dailyDedication, id, startDate, isConcluded);
    }

    @Override
    public String toString() {
        String status = isConcluded ? "Concluded" : "Not Concluded";
        return MessageFormat.format("[{3}] Habit {0}: {1}, {2}", id, name, motivation, status);
    }

    public boolean getIsConcluded() {
        return isConcluded;
    }

    public void setIsConcluded(Boolean isConcluded) {
        this.isConcluded = isConcluded;
    }

    public LocalDateTime getStartDate() {
        return startDate;
    }

```

 Copiar código



```
public void setIsConcluded(Boolean isConcluded) {
    this.isConcluded = isConcluded;
}

public LocalDateTime getStartDate() {
    return startDate;
}

public Integer getId() {
    return id;
}

public String getName() {
    return name;
}

public String getMotivation() {
    return motivation;
}

public void setMotivation(String motivation) {
    this.motivation = motivation;
}

public LocalTime getDailyDedicationTime() {
    return dailyDedicationTime;
}

public void increaseDailyDedicationTime(Integer minutes) {
    this.dailyDedicationTime = dailyDedicationTime.plusMinutes(minutes);
}
```

 Copiar código



```
        return id;
    }

    public String getName() {
        return name;
    }

    public String getMotivation() {
        return motivation;
    }

    public void setMotivation(String motivation) {
        this.motivation = motivation;
    }

    public LocalTime getDailyDedicationTime() {
        return dailyDedicationTime;
    }

    public void increaseDailyDedicationTime(Integer minutes) {
        this.dailyDedicationTime = dailyDedicationTime.plusMinutes(minutes);
    }

    public void decreaseDailyDedicationTime(Integer minutes) {
        this.dailyDedicationTime = dailyDedicationTime.minusMinutes(minutes);
    }

    public long getDaysDifference() {
        LocalDateTime ldt = LocalDateTime.now();
        Duration duration = Duration.between(this.startDate, ldt);
        return duration.toDays();
    }
}
```

 Copiar código



Explicações das Alterações:

1. **Método `create`**: Este método estático foi adicionado à classe `Habit` para encapsular a lógica de criação do objeto. Agora, a criação de um `Habit` com todos os parâmetros necessários é simplificada. Ele recebe os parâmetros e retorna um novo objeto `Habit` com eles.
2. **Construtores**: Os construtores da classe `Habit` permanecem para manter a flexibilidade, permitindo a criação de objetos com ou sem tempo diário de dedicação e outras informações.

Classe `HabitTracker` Atualizada

Agora, no método `addHabit`, chamamos `Habit.create` para criar o objeto `Habit`:

```
java Copiar código

public int addHabit(String name, String motivation, Integer dailyMinutesDedication,
                    Integer year, Integer month, Integer day, Integer hour, Integer minute) {
    Habit habit = Habit.create(name, motivation, dailyMinutesDedication, dailyHoursDedication,
                               year, month, day, hour, minute);
    this.habits.add(habit);
    int response = nextId;
    this.tracker.put(nextId, new ArrayList<>());
    this.nextId++;
    return response;
}
```

Agora, ao adicionar um hábito, o código fica mais limpo, com a responsabilidade de criar um novo `Habit` centralizada no método `create` da própria classe `Habit`.

Se precisar de mais alguma alteração ou explicação, estou à disposição! 😊

🔊 📄 👍 🗑️ ↺



// Método estático para criar Habit de maneira mais simples